World Action Models: From DreamZero to Dream-1

Training, Efficient Inference, Cross-Embodiment Transfer and Deployment

2026-10-30

Contents

1. What Is a World Action Model? . . . 2
2. How Are WAMs Trained? . . . 3
3. DreamZero: Joint Generation and Closed-Loop Control . . . 4
4. Dream-1: Zero-Shot Cross-Embodiment Transfer and Its Limits . . . 5
5. Efficient WAMs: How Much Future Information Is Needed at Test Time? . . . 6
6. VLA vs. WAM: Fair Comparisons and Model Selection . . . 6
7. Real-Robot Deployment, Contact Manipulation, and Research Directions . . . 8
References . . . 9

Course home ·简体中文 ·PDF

Item	Details
Topic	World Action Models
Revision	2026-10-30
Prerequisites	Imitation learning, Flow Matching and introductory robot control
License	CC BY-NC-SA 4.0 ·Xiong Zeng and contributors

1. What Is a World Action Model?

The central question: A robot must do more than recognize what is present. It must choose actions that bring about a desired change in the scene. A **World Action Model (WAM)** connects temporal changes in video with executable robot actions, mapping observations, proprioception, and language instructions to control. This lecture treats **WAMs as robot policies**.

1.1 From direct action prediction to future-aware learning

Let o_t denote camera observations, q_t robot state, ℓ a language instruction, $h_t = (o_{t-k:t}, q_{t-k:t})$ the observation history, and $A_t = (a_t, \dots, a_{t+H-1})$ an action chunk. A typical direct-action policy is

$$A_t \sim \pi_\theta(A \mid h_t, \ell).$$

The additional learning signal in a WAM comes from the **relationship between actions and future visual changes**. For a jointly generative video-action WAM, one can write

$$(O_t^+, A_t) \sim p_\theta(O^+, A \mid h_t, \ell), \qquad O_t^+ = (o_{t+1}, \dots, o_{t+F}).$$

Important qualification: This describes one family of WAMs, not a universal definition. A WAM may predict the future during training yet output only actions at inference, or reuse future-conditioned latent features without rendering frames. A visually plausible future does not guarantee correct contact forces, friction, or reachability.

1.2 Four common modeling patterns

Pattern	Inference pathway	Main benefit	Key cost or risk
Imagine-then-execute	Imagine candidate futures, then choose actions	Interpretable candidate outcomes	Expensive video sampling; imagined motion may be infeasible
Joint video-action	Jointly sample future visual content and action chunks	Tightly coupled temporal and action information	Expensive joint denoising
Video-feature-conditioned	Act using intermediate video-model representations	Pixel decoding may be unnecessary	Features may miss contact details
Auxiliary video prediction	Predict futures in training; keep action inference only	Lower latency and easier deployment	Future conditioning may be lost at test time

These are not mutually exclusive product categories. A system may combine joint training, compressed future representations, and a lightweight action head. **The defining research question** is what future-related supervision contributes to control, not whether the inference system renders visible video. [1, 3]

1.3 Three distinctions to keep clear

- **Future prediction $\neq$ physical correctness.** RGB alone often cannot reveal grip force, static-friction limits, or small slips.
- **A shared action interface $\neq$ cross-embodiment success.** Shared end-effector (EEF) poses and inverse kinematics (IK) make high-level commands portable but do not create a feasible path or a new skill.
- **Action-chunk duration $\neq$ model latency.** Returning a chunk covering 1.6 s of motion says nothing about whether inference itself takes 1.6 s, or whether the policy can replan every 20 ms.

Takeaway: The purpose of a WAM is to exploit temporal visual priors to improve generalization and action quality, not merely to synthesize realistic videos.

2. How Are WAMs Trained?

2.1 Data must link what was observed to what was executed

A robot demonstration typically contains aligned $(o_t, q_t, a_t, o_{t+1}, \ell, \Delta t)$ tuples; reward-based online interactions can additionally provide (r_t, d_t). Camera timestamps, action-execution times, control rates, coordinate frames, units, and controller modes must be explicit. Otherwise, a scene change may be paired with the wrong action.

Data source	What it contributes	What remains missing
Human or web video	Motion and interaction priors; task timing	Ground-truth executable robot actions
Robot teleoperation	Aligned video, proprioception, and actions	Breadth of tasks and embodiments
Simulation rollouts	Scalable actions and perturbations	Geometry, contact, and rendering realism
Online real-robot data	Failures, recovery, distribution correction	Cost, safety, and data governance

Video-only data can train visual prediction, but **unobserved robot actions must not be treated as ground-truth labels**. Pseudo-actions inferred by inverse dynamics or retargeting should be labeled as estimates. Multi-embodiment training must also normalize action semantics: base versus EEF frames, absolute versus relative poses, gripper commands, and timing. [1]

2.2 From video pretraining to robot actions

A common route starts with a pretrained video Diffusion Transformer (DiT), then attaches robot-state encoders and action encoders/decoders to align video latents z with action chunks A. Training may combine behavior cloning, action Flow Matching, video prediction, and cross-modal alignment. **There is no single loss used by all WAMs.**

For an introductory view of generative action learning, let $x^\star$ denote a target action vector or a joint vector of video latents and actions, and let $\epsilon \sim \mathcal{N}(0, I)$. One standard straight-path Flow Matching objective is

$$x_\tau = (1-\tau)\epsilon + \tau x^\star, \qquad \mathcal{L}_{\text{FM}} = \mathbb{E}\left[\left\| v_\theta(x_\tau, \tau, c) - (x^\star - \epsilon) \right\|_2^2\right],$$

where $\tau \in [0, 1]$ is **generative time**, not the robot's physical time t, and c contains image history, language, and proprioception. DreamZero learns a joint video-action velocity field; Fast-WAM asks how much of the benefit of video supervision can be retained in an action-only deployment path. [1, 3]

2.3 Three deployment pathways

The shared workflow is **record aligned trajectories → learn visual motion priors and action mappings → observe the real robot → produce an action chunk → execute part of the chunk → observe and correct again**. The main architectural choice is how much future information survives at test time:

1. **Explicit futures**: Generate candidate future images or latents, then infer actions. This can support interpretation or candidate evaluation.
2. **Joint futures and actions**: Sample future video latents and actions together; refresh the next prediction using new real observations.
3. **Actions only, or compressed future features**: Predict actions directly from trained temporal representations, optionally conditioning on a small number of future features.

Takeaway: Accurate alignment and action semantics are prerequisites for WAM training. Whether test-time future generation is worthwhile must be decided by measured robustness and latency.

3. DreamZero: Joint Generation and Closed-Loop Control

DreamZero (Ye et al., 2026) offers an unusually concrete public path into WAMs through its paper, model releases, and implementation. It starts from a pretrained video generator and turns video prediction and action generation into a coupled task. [1, 2]

3.1 Why joint generation is not "generate video, then run IK"

DreamZero builds on **Wan2.1-I2V-14B-480P** and conditions on language, image history, and robot state while jointly modeling future video latents and action chunks. Let $x_k^\star = [z_k^\star, A_k^\star]$ be the clean target for temporal chunk k. A teaching-level abstraction of joint velocity matching is

$$\mathcal{L}_{\text{joint}} = \mathbb{E}\Big[\|u_\theta(x_{k,\tau}, \tau; h_k, \ell, q_k) - (x_k^\star - \epsilon_k)\|_2^2\Big].$$

This equation is a **pedagogical abstraction**, not a verbatim transcription of every weighting term in the paper. Video and action are coupled inside the generative process, rather than implemented as an external cascade of fully rendered video and IK. Causal chunking and observed history allow the model to incorporate new real observations over successive control rounds. [1]

3.2 Training scale, clocks, and what "zero-shot" means

The AgiBot experiments use approximately **500 hours, 7,200 trajectories, and 22 environments**. A representative setup has video near 5 FPS, low-level actions near 30 Hz, and chunks of $H = 48$ actions. The chunk therefore covers about $48/30 = 1.6$ s of robot motion. That is **not** the model's inference time. DreamZero needs a diverse robot dataset; it cannot be trained from scratch using only thirty minutes of data. [1]

DreamZero reports markedly better **task progress** than its evaluated strong VLA baselines on unseen-task and unseen-environment settings. Task progress should not be silently relabeled as full-task success. It also presents two different kinds of transfer evidence: adding videos from humans or other robots as a learning signal, and **adapting to the YAM robot using about 30 minutes of play data** before testing generalization to new tasks. The latter is **few-shot embodiment adaptation**, not zero-data transfer to an unseen robot. [1]

3.3 What lies behind 150 ms and 7 Hz?

Naive joint sampling of a large video model is costly. The paper uses asynchronous inference and execution, parallel computation, caching, and kernel optimizations. It also introduces **DreamZero-Flash**, which changes the training distribution of video and action noise levels so that useful actions can be predicted even when the future-video latent is still noisy.

Setting in the same paper/task	Denoising steps	Task progress	Latency per chunk
DreamZero	4	83%	350 ms
DreamZero, naively reduced to one step	1	52%	150 ms
DreamZero-Flash	1	74%	150 ms

The comparison shows that **fewer sampling steps can reduce control quality unless the training procedure changes**. With further optimizations the paper reports about **150 ms per chunk and 7 Hz high-level closed-loop control on two GB200 GPUs**. These are measurements under a particular hardware and systems configuration, not guarantees for a single RTX 4090 or other tasks. [1]

A genuine closed loop must periodically replace imagined context with **real camera and proprioceptive observations**. A causal KV cache can reuse history but cannot substitute for contact feedback or remove state-estimation errors.

Takeaway: DreamZero demonstrates that joint generative WAMs can be engineered for real-robot feedback control. Generalization metrics, clock rates, and hardware conditions must always accompany performance claims.

4. Dream-1: Zero-Shot Cross-Embodiment Transfer and Its Limits

Dream Labs introduced Dream-1 and a cloud API in **October 2026**. The company demonstrates zero-shot control on several previously unseen bimanual robots with parallel grippers. As of this revision, the public evidence is mainly an **official technical announcement and product demonstrations**, not a DreamZero-style disclosure of full weights, training recipes, and architecture ablations. It is important to separate observed demonstrations, plausible mechanisms, and causal conclusions that have not yet been established. [4]

4.1 A common action language for different robot bodies

For supported native embodiments, Dream-1 can output joint controls; it can also predict target EEF poses and gripper open/close commands for the two arms. A teaching-level interface for robot r is

$$A_t^{\text{shared}} = \{(T^{\star}_{\text{EE},i}, g_i)\}_{i\in\{L,R\}}, \qquad q^{\star}_{t+1} = \text{IK}_r(A_t^{\text{shared}}; q_t, \mathcal{C}_r).$$

This is **not** a claim about Dream-1's exact internal action tensor. The IK solver produces a joint-space target; trajectory constraints, collision checks, and a low-level controller are still required. The shared EEF interface reduces robot-specific joint-space differences, but IK alone neither teaches new tasks nor guarantees feasible motions. A VLA can also use a shared EEF action space. [4]

4.2 Why might visual priors transfer across embodiments?

The official description uses **three camera streams**: **one head view and one wrist view per arm**. The head view provides global scene and body context; wrist views reveal local manipulation. Robots have different joints and link geometries, but many visual patterns of approaching, grasping, moving, and placing objects may be shared across bodies. Three complementary hypotheses help explain the results: a **shared EEF/IK control interface**, **video-action temporal supervision**, and **cross-embodiment visual representations**.

The independent contribution of each factor still requires controlled ablations using matched cameras, action spaces, training data, and controllers. Dream-1's cross-embodiment results do **not** imply that only WAMs can transfer without target-robot fine-tuning: the VLA approach LAP-3B also reports zero-shot transfer to unseen robots. [4, 8]

4.3 Zero-shot transfer does not eliminate post-training

Generalization level	What it means	The relevant distinction
Unseen tasks/environments	Robot may be seen; task or scene is new	Both projects report related results
Few-shot embodiment adaptation	Update the policy after collecting target-robot data	DreamZero's roughly 30-minute YAM result
Zero-shot embodiment transfer	Attempt execution without target-robot training data	Demonstrated by Dream-1 for some bimanual grippers

The Dream-1 announcement discusses IK quality, joint limits, workspace, and dynamical mismatch, and offers Fine-tuning-as-a-Service (FaaS). It presents **five-minute adaptation examples** for some embodiments; notably different dexterous-hand setups require more data in the reported examples rather than demonstrating identical zero-shot behavior. [4]

The API returns chunks covering about **1.6 seconds of actions** and publishes an action-hour pricing scheme. The announcement does not fully specify reproducible end-to-end network $p95$ latency, device-side caching, or low-level command frequency. **DreamZero's 150 ms figure must not be reused as Dream-1 API latency**. [4]

Takeaway: Dream-1 illustrates the potential of coupling a shared robot interface to temporal visual priors. Current public evidence does not establish that every WAM beats every VLA, nor that unseen dexterous hands can be deployed reliably without adaptation.

5. Efficient WAMs: How Much Future Information Is Needed at Test Time?

A central deployment trade-off is that **future information may improve robustness, while video processing and action sampling can increase latency**. Fast-WAM and two distinct papers that both use the name Faster-WAM explore different points on this frontier. [3, 5, 6]

5.1 Fast-WAM: Co-train on video, infer actions only

Fast-WAM (Yuan et al., 2026) keeps **video-prediction supervision during training** but uses the video backbone as a representation extractor during inference, without generating future frames. In the authors' ablations, removing video co-training often hurts more than removing explicit test-time future generation. The paper reports around **190 ms latency** and a **greater than four-fold speedup** over its evaluated future-generation baselines under its stated hardware and workloads. [3]

It answers whether **much of the benefit of future prediction can be captured during training**. It does not prove that future-conditioned features are useless at inference in all environments.

5.2 Faster-WAM (Ma et al.): A shallow action expert

Faster-WAM: Do World Action Models Need Deep Action Modules? (arXiv:2608.02365) leaves most representational capacity in the video backbone and connects a **single-layer action expert** to multi-layer video features using **Lite KV-Fusion**. The project reports approximately **129 ms** on a particular real-robot deployment workstation versus **502 ms** for its Fast-WAM implementation under the same setup. These measurements cannot be directly ranked against numbers from other hardware. The target bottleneck is **action-head depth and redundant computation**. [5]

5.3 Faster-WAM (Zhao et al.): Sparse future conditioning

Faster-WAM: Efficient Inference-Time Future Conditioning for Robust World Action Models (arXiv:2608.04404) asks whether removing future representations entirely at inference sacrifices out-of-distribution robustness. Its **SparseMoT** and **Interval KV-Fusion** designs reuse future features at only a few selected points.

Under that paper's LIBERO-Plus protocol, the authors report **73.57%** success against **49.14%** for their Fast-WAM baseline, while running **2.21 times faster** than their Joint-WAM implementation. The result supports retaining compressed future conditioning in some shifted settings; it is **not** a cross-paper, cross-hardware performance leaderboard. [6]

Method	What survives at inference?	Primary bottleneck addressed
DreamZero-Flash	Joint video-action sampling with fewer generative steps	Cost of joint generation
Fast-WAM	Learned world features; principally an action inference path	Redundant test-time future generation
Faster-WAM (Ma)	Multi-depth video features and one-layer action head	Overly deep action experts
Faster-WAM (Zhao)	Sparse future features during action denoising	OOD robustness versus latency

Takeaway: "No rendered future video" does **not** mean "no future-related information." The design decision is how to allocate computation across **action-only inference**, **compressed future conditioning, and full joint generation**.

6. VLA vs. WAM: Fair Comparisons and Model Selection

6.1 Compare learning signals rather than labels

Dimension	Typical direct-action VLA	Representative WAM variants
Training targets	Observation/language-to-action supervision	Actions plus future-related visual supervision
Starting priors	VLM semantic and visual representations	Video-model temporal and motion priors
Output at execution	Discrete commands or continuous action chunks	Still executable robot action chunks
Runtime cost	Backbone, action head, sampling steps	Also depends on retained future features/video
Generalization hypothesis	Task semantics and learned action distribution	Motion consequences and temporal interactions
Failure modes	Action coverage, perception error	Plus visual hallucinations and added latency

It is inaccurate to say "VLA always acts in one pass, whereas WAM always denoises video." Flow-matching VLAs such as π_0 and $\pi_{0.5}$ also use iterative action generation. It is equally incorrect to say "VLAs need fine-tuning but WAMs do not." Embodiment coverage, action semantics, camera layouts, and control interfaces all matter. LAP-3B provides an important counterexample through its reported zero-shot cross-embodiment results. [8]

6.2 How to account for compute and latency

Inference memory includes model weights, activations, KV caches, visual latents, and runtime workspace. With P resident parameters and b bits per parameter, a **theoretical lower bound for weight storage** is

$$M_{\text{weights}}\ [\text{GB}] \approx \frac{Pb}{8 \times 10^9}, \qquad M_{\text{peak}} > M_{\text{weights}}.$$

In decimal units, BF16 weights take about 2 GB per billion parameters. This is **not** a GPU memory recommendation. Video frame count, resolution, sequence length, denoising steps, quantization, fused attention, and parallelism all change realized cost. Training sets, pretraining budgets, accelerators, and outcome metrics differ too much to justify a universal claim that WAMs cost a fixed multiple of VLAs.

A deployment latency budget should at least include

$$T_{\text{e2e}} = T_{\text{sense}} + T_{\text{encode}} + T_{\text{network}} + T_{\text{model}} + T_{\text{dispatch}}.$$

Report $p50/p95/p99$ latency, concurrent load, action duration, and actual replanning rate. Accelerating the network does not necessarily accelerate the end-to-end robot loop by the same factor.

6.3 A practical selection procedure

- **Abundant robot action data, limited GPUs, and strict timing**: Start with a direct-action VLA or flow-policy baseline. Then introduce video co-training under the same data budget to measure its marginal value.
- **Difficult motion changes and distribution shift**: Compare Fast-WAM with a WAM that retains compressed future conditioning. Focus on shifted-task success and recovery rather than video quality alone.
- **Need to inspect candidate outcomes**: Consider full joint future sampling only when visual foresight justifies the latency and closed-loop cost.
- **Large embodiment changes**: Normalize action frames and low-level interfaces first; evaluate zero-shot behavior separately so IK improvements are not incorrectly credited to video prediction.

To isolate the benefit of future supervision, hold the robots, tasks, controllers, cameras, action spaces, data, and evaluation

budget fixed. Change only future-related training signals and the amount of future conditioning retained at inference.

Takeaway: VLAs and WAMs are comparable and can be combined. Select the design based on **out-of-distribution reliability, latency, data, and deployment interfaces** rather than an assumed architectural winner.

7. Real-Robot Deployment, Contact Manipulation, and Research Directions

7.1 Connect model time to robot time

There are three different time scales to report:

1. **Low-level control rate** f_{ctrl}: How often joint or EEF commands actually update.
2. **Policy feedback rate** f_{policy}: How often the model incorporates new real observations and replans.
3. **Action-chunk coverage** H/f_{ctrl}: The duration of motion represented by one predicted chunk.

A layered deployment path is **synchronized cameras and state** $\rightarrow$ **WAM inference** $\rightarrow$ **safety filtering/IK** $\rightarrow$ **low-level controller** $\rightarrow$ **new observations**. With asynchronous inference, chunks need timestamps and expiration rules, smooth transitions when replacements arrive, and a safe stop on timeout or connection loss.

7.2 Visual imagination alone cannot solve contact control

Precision manipulation fails for reasons such as occluded object-pose errors, inference delays at contact onset, friction/compliance mismatch, finger slip, and IK solutions that are kinematically reachable yet collision-prone. Video prediction can provide a high-level prior about how an object should move, but cannot replace **force/tactile feedback, impedance or compliant control, actuator limits, and collision checks**.

Moving from bimanual parallel grippers to **multifingered dexterous hands** changes action dimension, contact mechanics, and low-level control substantially. A reasonable research architecture is: a WAM proposes targets and manipulation intent; state estimation closes the visual loop; a low-level controller handles contact stability; demonstration adaptation, residual control, or RL post-training repairs remaining errors. This is an **engineering hypothesis**, not a universally verified Dream-1 recipe.

7.3 What evidence makes a WAM result convincing?

Evaluation target	Minimum reporting standard
Task generalization	Seen vs. unseen tasks; full success vs. task progress
Embodiment generalization	Whether target robot entered pretraining; adaptation data used
Temporal robustness	Camera perturbations, occlusion, object shifts, contact disturbances
Compute	Same-hardware end-to-end latency, $p95$, memory, throughput, sampling steps
Executability	Violations, IK failures, collisions, slips, and recovery rate
Reproducibility	Weights/code, data protocol, controller, trial counts, variability

A particularly informative ablation uses the same backbone, robot, and data to compare **no video supervision** $\rightarrow$ **video supervision during training** $\rightarrow$ **compressed test-time future conditioning** $\rightarrow$ **full joint future generation**. This isolates what video-derived future information actually contributes to robot behavior better than comparing isolated best numbers across papers.

Three final messages:

1. WAMs are valuable for **future-aware action representations and temporal supervision**, not for pretty videos.
2. DreamZero supports joint generation and optimized closed-loop control; Dream-1 demonstrates promising transfer to some unseen bimanual grippers, with a different level of public evidence.

3. The efficient-WAM question is not "world model or no world model," but **how much future information a robot needs at inference under a real latency budget**.

References

[1] Ye et al., *World Action Models are Zero-shot Policies* (DreamZero), 2026. Primary source for joint modeling, training, optimization, and real-robot results.

[2] NVIDIA GEAR, DreamZero project and released code, 2026.

[3] Yuan et al., *Fast-WAM: Do World Action Models Need Test-time Future Imagination?*, 2026.

[4] Dream Labs, *Zero-Shot to New Robots: Introducing Dream-1*, Oct. 2026. **Official announcement; not a full, publicly reproducible peer-reviewed technical paper.**

[5] Ma et al., *Faster-WAM: Do World Action Models Need Deep Action Modules?*, 2026.

[6] Zhao et al., *Faster-WAM: Efficient Inference-Time Future Conditioning for Robust World Action Models*, 2026. **A different paper that shares the name Faster-WAM with [5].**

[7] Su et al., *1Step-WAM: Your World Action Model but Faster using 1-Step Action Generation*, 2026. Further reading on one-step action generation.

[8] Zha et al., *LAP: Language-Action Pre-training Enables Zero-Shot Cross-Embodiment Transfer*, RSS 2026. Counterexample showing that a VLA can also transfer zero-shot to new robots.

Equations marked as teaching abstractions are not assertions about an unpublished model's exact implementation.